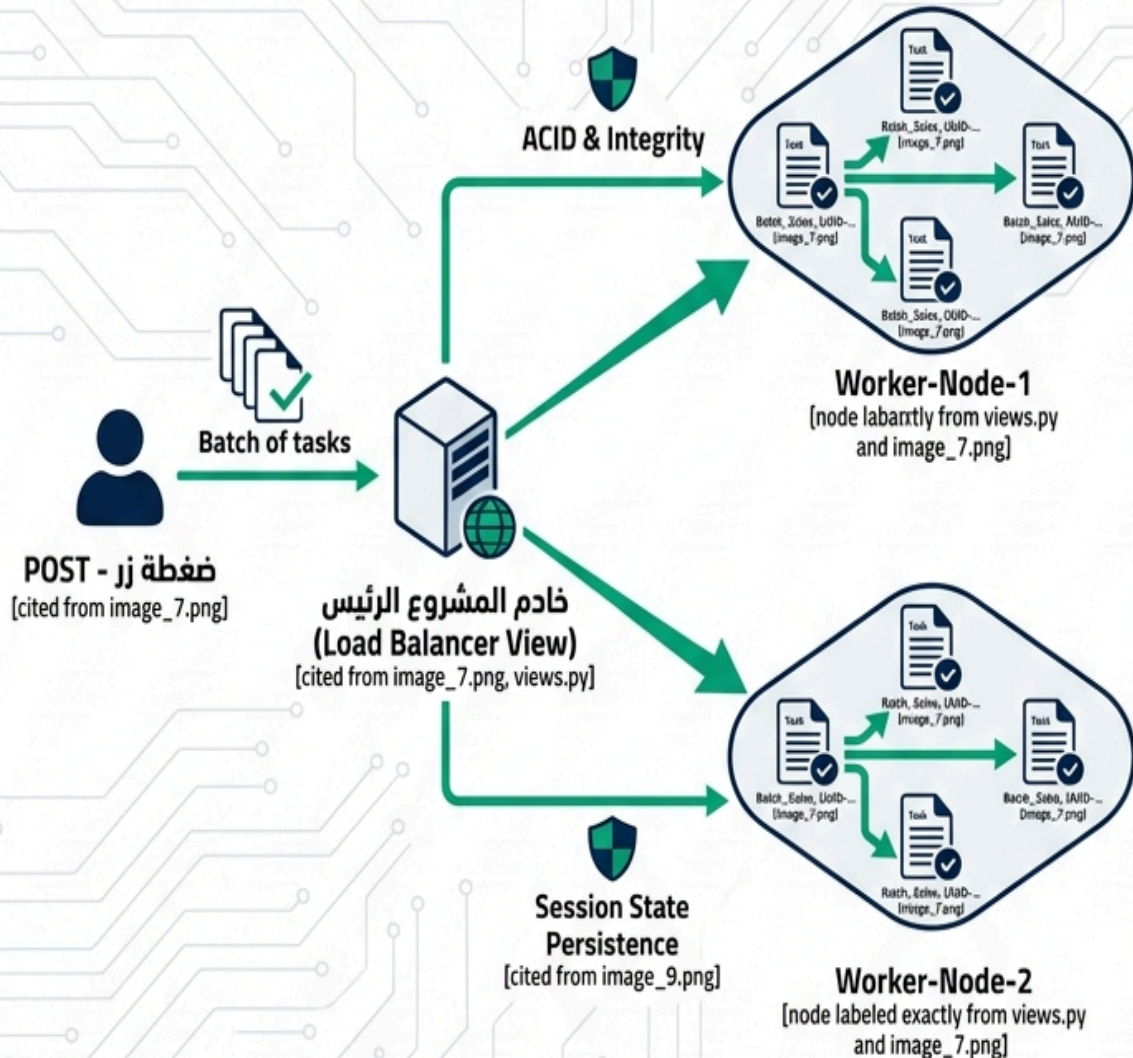


تقرير مشروع الأنظمة الموزعة

محاكاة المعالجة المتوازية وتوزيع الأحمال



جامعة دمشق

كلية الهندسة المعلوماتية

إعداد الطلاب:

- دعاء حسن يونس
- محمد غزوان ديب
- سيدرا منذر هلال
- سيدرا كمال
- محمد قاسم التنبير

قسم: الذكاء الصناعي - السنة الرابعة.

إشراف المهندس: حذيفة عقيل.

1. مقدمة عن المشروع (Introduction)

تعتبر مشكلة معالجة كميات البيانات الضخمة (Batch Processing) وتوزيعها عبر العقد البرمجية من أهم التحديات التي تواجه الأنظمة البرمجية الحديثة القابلة للتوسع (Scalable Systems). في معماريات الويب التقليدية، يؤدي تنفيذ العمليات الكثيفة حسابياً بشكل متزامن وتتابعي (Sequential) على السيرفر الرئيسي إلى حدوث اختناقات برمجية وبطء شديد في زمن الاستجابة (Response Time)، مما ينعكس سلباً على تجربة المستخدم النهائي وقد يؤدي أحياناً إلى انهيار خادم الويب تحت الضغط العالي.

يأتي هذا المشروع كحل عملي وتطبيقي لمحاكاة معمارية الأنظمة الموزعة عبر بناء لوحة تحكم ذكية لإدارة مبيعات ومخازن المتاجر الضخمة (Smart Sales Dashboard). حيث يعتمد النظام على استقبال دفعات البيانات الكبيرة وتجزئتها، ومن ثم توجيهها وإدارة معالجتها بشكل منفصل وموزع بالخلفية لضمان (Sales Inventory Batch) استقرار النظام الأساسي وخفته.

2. أهداف المشروع البرمجية (Project Objectives)

- محاكاة المهام غير المتزامنة (Asynchronous Tasks Simulation): فصل واجهة المستخدم والعمليات الحسابية الخفيفة عن السيرفر الرئيسي عبر إرسال المعالجات الثقيلة إلى الخلفية.
- تطبيق خوارزمية موازن الأحمال (Load Balancing): توزيع دفعات المعالجة بشكل عادل ومتناوب لضمان الاستغلال الأمثل لموارد الخوادم المتاحة.
- إدارة الجلسات اللحظية (Session-Based State Management): بناء مخزن محلي لحفظ حالة الدفعات وضمان عدم ضياع البيانات عند تحديث المتصفح.
- تطوير واجهة تفاعلية ديناميكية (Dynamic UI): عرض نتائج التوزيع وتوليد المعرفات الفريدة لحظة بلحظة وبصورة برمجية منظمة.

تم تصميم النظام الحالي وبناء المنطق البرمجي (Core Logic) بالكامل ضمن الطبقة الخلفية للإطار البرمجي ومربوطاً بالـ Sessions كنموذج أولي ومسوّدة برمجية متكاملة (Simulation Proof-of-Concept) لضمان صحة تدفق البيانات وتوزيعها قبل الانتقال لربط خوادم ميديوم خارجي حقيقي مثل Redis و Celery في المراحل القادمة.

3. البيئة البرمجية والتقنيات المستخدمة (Technical Stack)

تم الاعتماد على التقنيات التالية:

- لغة البرمجة: لموثوقيتها في معالجة البنى المنطقية المعقدة Python.
 - إطار العمل الرئيسي: وإدارة الجلسات وإدارة معمارية الويب والـ Django Framework URLs بمرونة.
 - الواجهة الأمامية: مخصصة ومصممة بأسلوب تفاعلي منظم يدعم التحديث وعرض الحالات HTML5 / CSS3.
-

4. بنية النظام اللوجستية (System Architecture & Logic)

يعتمد السيناريو البرمجي في الكود على استقبال طلب المعالجة من المستخدم عبر الـ Dashboard، حيث يقوم الـ View باستلام الطلب الذي يحتوي على دفعة مبيعات معينة، ويقوم فوراً وبشكل ديناميكي بتطبيق خوارزمية Round-Robin (التناوب الدائري).

يتم توزيع الأسطر الناتجة بالتناوب التام بين عقدتي معالجة وهميتين تم إعدادهما منطقياً داخل الكود وهما:

1. worker-node-1 (العقدة البرمجية الأولى)

2. worker-node-2 (العقدة البرمجية الثانية)

يتم إعطاء كل مهمة معالجة معرف فريد من نوعه (Task UUID)، وتُحفظ هذه البنية اللحظية داخل الـ Session الخاص بالمتصفح لضمان استقرار البيانات (Persistence) ومنع زوالها عند إعادة تحميل الصفحة.

الخطة التطويرية للمرحلة القادمة (Roadmap):

6. استراتيجية التخزين المؤقت (**Distributed Caching**): دمج طبقة تخزين مؤقت (مثل Redis) لتخزين المنتجات الأكثر طلباً وتقليل الاستعلامات المباشرة من قاعدة البيانات.
7. التحكم في الأقفال (**Concurrency Control**): تطبيق مفهوم القفل المتفائل (Optimistic Locking) أو التشاؤمي (Pessimistic) عند تحديث كميات المخزون الحساسة.
8. سلامة المعاملات (**Transaction Integrity / ACID**): ضمان أن العمليات المركبة (دفع + تحديث مخزون + إنشاء طلب) تنجح كلها أو تفشل كلها، حتى تحت الوصول المتزامن.
9. اختبار الاستقرار تحت الضغط (**Stress Testing**): تقديم تقرير يثبت قدرة النظام على تخدم 100 مستخدم متزامن على الأقل دون انهيار أو فقدان بيانات.
10. القياس وتحديد الاختناقات (**Benchmarking & Bottleneck Analysis**): قياس زمن الاستجابة لأهم العمليات، تحديد اختناق واحد على الأقل، وتقديم مقارنة رقمية قبل التحسين وبعده.

ملف `views.py` البنية المنطقية المعتمدة لاستقبال دفعات البيانات وتوزيع الأحمال ومعالجة الأخطاء الاستثنائية:

Python

```
import uuid
```

```
from django.shortcuts import render
```

```
from django.contrib import messages
```

```
def process_batch_view(request):
```

```
    # التأكد من تهيئة الجلسة لحفظ المهام السابقة إذا وجدت
```

```
    if 'tasks_history' not in request.session:
```

```
        request.session['tasks_history'] = []
```

```
if request.method == 'POST':
```

```
    try:
```

```
        # محاكاة استقبال دفعة بيانات للمبيعات والمخازن
```

```
        # حجم الدفعة الافتراضي لكل كبسة زر = 5
```

```
        batch_size = 5
```

```
        current_history = request.session['tasks_history']
```

```
        for i in range(batch_size):
```

```
            # تطبيق خوارزمية موازن الأحمال بالتناوب الدوري (Round-Robin)
```

```
            # إذا كان الترتيب زوجي يذهب للعقدة الأولى، وإذا كان فردي يذهب للعقدة
```

الثانية

```
            assigned_node = 'worker-node-1' if i % 2 == 0 else  
            'worker-node-2'
```

```
            # توليد معرف فريد حقيقي لكل مهمة معالجة داخل الدفعة
```

```
            task_id = str(uuid.uuid4())
```

```
            # بناء هيكل البيانات الخاص بالمهمة المحاكاة
```

```
            task_data = {
```

```
                'task_id': task_id,
```

```
                'status': 'Success',
```

```
                'node': assigned_node,
```

```
                'batch_name': f"Sales_Batch_{task_id[:8].upper()}"
```

```
            }
```

إدراج المهمة الجديدة في أعلى القائمة لتظهر في المقدمة #

```
current_history.insert(0, task_data)
```

حفظ التغييرات داخل الجلسة وتحديث الـ State

```
request.session['tasks_history'] = current_history
```

```
request.session.modified = True
```

تمت معالجة دفعة البيانات وتوزيعها بنجاح" messages.success(request, "
عالي")

except Exception as e:

معالجة استثنائية لحماية السيرفر من الانهيار في حال حدوث أي خلل

فشلت المعالجة أثناء توزيع الأحمال" f" messages.error(request, "
{str(e)}")

```
return render(request, 'dashboard.html', {
```

```
'tasks': request.session.get('tasks_history', [])
```

```
})
```

حفظ التغييرات داخل الجلسة وتحديث الـ State

```
request.session['tasks_history'] = current_history
```

```
request.session.modified = True
```

```
messages.success(request, "تمت معالجة دفعة البيانات وتوزيعها بنجاح"  
!)عال
```

```
except Exception as e:
```

```
# معالجة استثنائية لحماية السيرفر من الانهيار في حال حدوث أي خلل
```

```
messages.error(request, f"فشلت المعالجة أثناء توزيع الأحمال"  
{str(e)})
```

```
return render(request, 'dashboard.html', {  
    'tasks': request.session.get('tasks_history', [])  
})
```